

`context.store` 是 Makers Agent 内置的对话存储能力，零配置即可保存与读取对话历史，并为 Claude / OpenAI / LangGraph / DeepAgents Agent 框架提供原生适配。

底层基于平台的 Blob 存储，Node 与 Python 两端 API 完全镜像（JavaScript `camelCase`、Python `snake_case`），框架接入章节以 **TypeScript** 为主，每节附 Python 等价示例。

## 在两种 Runtime 中使用

`store` 在 `agents/` 与 `cloud-functions/` 两类目录的代码中指向同一份数据。

| 目录                            | 使用入口                             | 典型用途                         |
|-------------------------------|----------------------------------|------------------------------|
| <code>agents/</code>          | <code>context.store</code>       | LLM 对话主路径，追加消息、Checkpointner |
| <code>cloud-functions/</code> | <code>context.agent.store</code> | 对话列表 API、消息查询                |

## 框架接入

`context.store` 同时提供框架原生适配器和通用 API。

| framework               | 推荐用法                                                | 入口（TS）                                                              |
|-------------------------|-----------------------------------------------------|---------------------------------------------------------------------|
| <code>claude-sdk</code> | 框架原生 <code>SessionStore</code>                      | <code>context.store.claudeSessionStore()</code>                     |
| <code>openai-sdk</code> | 框架原生 <code>Session</code>                           | <code>context.store.openaiSession(sessionId)</code>                 |
| <code>langgraph</code>  | 框架原生 <code>Checkpointner + BaseStore</code>         | <code>context.store.langgraphCheckpointner / .langgraphStore</code> |
| <code>deepagents</code> | 复用 <code>LangGraph Checkpointner + BaseStore</code> | <code>context.store.langgraphCheckpointner / .langgraphStore</code> |
| <code>crewai</code>     | 暂不支持，使用通用 API 自管                                    | -                                                                   |

### Claude Agent SDK

`claudeSessionStore()` 返回 `SessionStore` 协议实现（`append` / `load` / `listSessions` / `delete` / `listSubkeys`）。

```
// typescript
const sessionStore = context.store.claudeSessionStore()
```

```
// python
session_store = context.store.claude_session_store()
```

## OpenAI Agents SDK

`openaiSession(sessionId, { maxItems })` 返回 OpenAI Agents SDK 的 `Session` 协议实现 (`getItems` / `addItem` / `popItem` / `clearSession`)。

```
// typescript
const session = context.store.openaiSession(context.conversation_id)

// python
session = context.store.openai_session(context.conversation_id)
```

## LangGraph

`langgraphCheckpoint` 实现 `BaseCheckpointSaver` (`getTuple` / `list` / `put` / `putWrites`), `langgraphStore` 实现 `BaseStore` (`get` / `put` / `search` / `listNamespaces` / `batch`), `BaseStore` 的 `search` 暂不支持按语义检索。

```
// typescript
const checkpointer = context.store.langgraphCheckpoint
const store = context.store.langgraphStore

// python
checkerpoint = context.store.langgraph_checkerpoint
store = context.store.langgraph_store
```

## DeepAgents

DeepAgents 基于 LangGraph 构建，直接复用 `langgraphCheckpoint` + `langgraphStore` 两个适配器。

```
// typescript
const checkpointer = context.store.langgraphCheckpoint
const store = context.store.langgraphStore

// python
checkerpoint = context.store.langgraph_checkerpoint
store = context.store.langgraph_store
```

## CrewAI

CrewAI 自带 Memory 强依赖向量库（默认 LanceDB + embedder），`context.store` 暂时未建向量层，对话历史用 `context.store` 通用 API 自管即可，多角色编排、任务链、工具调用等核心能力完全不受影

响。

## 通用 API

### 方法总览

方法均挂在 `context.store` 上。Node 端使用对象解构入参（`camelCase`），Python 端使用关键字参数（`snake_case`），语义完全一致。

| Node 方法                          | Python 方法                          | 描述                                  |
|----------------------------------|------------------------------------|-------------------------------------|
| <code>appendMessage</code>       | <code>append_message</code>        | 追加一条消息；对话不存在时自动创建                   |
| <code>getMessages</code>         | <code>get_messages</code>          | 拉消息列表，支持游标分页                        |
| <code>updateMessage</code>       | <code>update_message</code>        | 覆盖式更新指定消息                           |
| <code>deleteMessage</code>       | <code>delete_message</code>        | 删除单条消息                              |
| <code>clearMessages</code>       | <code>clear_messages</code>        | 清空消息但保留对话元信息                        |
| <code>getConversation</code>     | <code>get_conversation</code>      | 取对话元信息                              |
| <code>listConversations</code>   | <code>list_conversations</code>    | 列对话，按 <code>lastMessageAt</code> 倒序 |
| <code>updateConversation</code>  | <code>update_conversation</code>   | 更新对话元信息，浅合并                         |
| <code>deleteConversation</code>  | <code>delete_conversation</code>   | 删除整个对话（不可恢复）                        |
| <code>toAnthropicMessages</code> | <code>to_anthropic_messages</code> | 把消息列表转成 Anthropic Messages 格式       |
| <code>toOpenAIInput</code>       | <code>to_openai_input</code>       | 把消息列表转成 OpenAI Chat Completions 格式  |

#### `appendMessage` / `append_message`

向指定对话追加一条消息，对话不存在时自动创建，并按需建立用户索引。

#### 参数

| Parameter                                                  | Type                                                        | Required | Description          |
|------------------------------------------------------------|-------------------------------------------------------------|----------|----------------------|
| <code>conversationId</code> / <code>conversation_id</code> | <code>string</code>                                         | Yes      | 业务侧对话 ID，长度 ≤ 256 字节 |
| <code>role</code>                                          | <code>'user' \   'assistant' \   'system' \   'tool'</code> | Yes      | 消息角色                 |

| Parameter        | Type                         | Required | Description                               |
|------------------|------------------------------|----------|-------------------------------------------|
| content          | string \  string[] \  object | Yes      | 消息正文，支持纯文本 / 字符串数组 / 多模态 dict，序列化后 ≤ 50MB |
| metadata         | Record<string, any>          | No       | 业务自定义字段（token 数、tool_call、来源标签等）          |
| userId / user_id | string                       | No       | 关联用户，传入后写入用户索引，便于按用户列对话                   |

返回值

新消息的 messageId / message\_id（形如 msg\_xxx）。

TS 示例：

```
const messageId = await context.store.appendMessage({
  conversationId: context.conversation_id,
  role: 'user',
  content: context.request.body.message,
  userId: context.request.body.userId,
  metadata: { source: 'web' },
})
```

Python 示例：

```
message_id = await context.store.append_message(
    conversation_id=context.conversation_id,
    role="user",
    content=context.request.body["message"],
    user_id=context.request.body.get("user_id"),
    metadata={"source": "web"},
)
```

getMessages / get\_messages

拉取指定对话的消息列表，支持游标分页。**对话不存在不抛异常**，返回空列表，默认按时间正序（order='asc'，最早优先），方便直接拼 prompt。

参数

| Parameter                                                  | Type                          | Required | Description                                                 |
|------------------------------------------------------------|-------------------------------|----------|-------------------------------------------------------------|
| <code>conversationId</code> / <code>conversation_id</code> | <code>string</code>           | Yes      | 对话 ID                                                       |
| <code>limit</code>                                         | <code>number</code>           | No       | 单页条数，默认 <code>20</code> ，范围 <code>[1, 100]</code>           |
| <code>order</code>                                         | <code>'asc' \   'desc'</code> | No       | 排序方向，默认 <code>'asc'</code> （最早优先）                           |
| <code>after</code>                                         | <code>string</code>           | No       | 游标，取该 <code>messageId</code> 之后的消息；与 <code>before</code> 互斥 |
| <code>before</code>                                        | <code>string</code>           | No       | 游标，取该 <code>messageId</code> 之前的消息；与 <code>after</code> 互斥  |

返回值

`list[Message]` —— 消息数组，对话不存在返回 `[]`。

TS 示例：

```
const messages = await context.store.getMessages({
  conversationId: context.conversation_id,
  limit: 50,
})
const reply = await openai.chat.completions.create({
  model: 'gpt-4o',
  messages: context.store.toOpenAIInput(messages),
})
```

Python 示例：

```
messages = await context.store.get_messages(
    conversation_id=context.conversation_id,
    limit=50,
)
reply = await openai_client.chat.completions.create(
    model="gpt-4o",
    messages=context.store.to_openai_input(messages),
)
```

**updateMessage / update\_message**

覆盖式更新一条消息，仅传入的字段会被覆盖，未传字段保持原值；`updatedAt` / `updated_at` 自动刷新。

参数

| Parameter                        | Type                         | Required | Description                |
|----------------------------------|------------------------------|----------|----------------------------|
| conversationId / conversation_id | string                       | Yes      | 对话 ID                      |
| messageId / message_id           | string                       | Yes      | 目标消息 ID                    |
| content                          | string \  string[] \  object | No       | 新正文；不传则保持原值                |
| metadata                         | Record<string, any>          | No       | 整体覆盖 metadata（非合并）；不传则保持原值 |

返回值

Message —— 更新后的完整消息对象。

TS 示例：

```
const updated = await context.store.updateMessage({
  conversationId: context.conversation_id,
  messageId: 'msg_abc123',
  content: 'corrected answer',
  metadata: { edited: true },
})
```

Python 示例：

```
updated = await context.store.update_message(
    conversation_id=context.conversation_id,
    message_id="msg_abc123",
    content="corrected answer",
    metadata={"edited": True},
)
```

deleteMessage / delete\_message

删除单条消息。

参数

| Parameter                        | Type   | Required | Description |
|----------------------------------|--------|----------|-------------|
| conversationId / conversation_id | string | Yes      | 对话 ID       |

| Parameter                                        | Type                | Required | Description |
|--------------------------------------------------|---------------------|----------|-------------|
| <code>messageId</code> / <code>message_id</code> | <code>string</code> | Yes      | 目标消息 ID     |

#### TS 示例:

```
await context.store.deleteMessage({
  conversationId: context.conversation_id,
  messageId: 'msg_abc123',
})
```

#### Python 示例:

```
await context.store.delete_message(
    conversation_id=context.conversation_id,
    message_id="msg_abc123",
)
```

### `clearMessages` / `clear_messages`

清空对话中所有消息，但保留 `ConversationMeta`，彻底删除请用 `deleteConversation`。

#### 参数

| Parameter                                                  | Type                | Required | Description |
|------------------------------------------------------------|---------------------|----------|-------------|
| <code>conversationId</code> / <code>conversation_id</code> | <code>string</code> | Yes      | 对话 ID       |

#### TS 示例:

```
await context.store.clearMessages({ conversationId: context.conversation_id })
```

#### Python 示例:

```
await context.store.clear_messages(conversation_id=context.conversation_id)
```

### `getConversation` / `get_conversation`

获取对话元信息。

#### 参数

| Parameter                                                  | Type                | Required | Description |
|------------------------------------------------------------|---------------------|----------|-------------|
| <code>conversationId</code> / <code>conversation_id</code> | <code>string</code> | Yes      | 对话 ID       |

返回值

`ConversationMeta` —— 包含 `conversationId` / `createdAt` / `lastMessageAt` / `messageCount` / `metadata` 等字段。

TS 示例:

```
const meta = await context.store.getConversation({
  conversationId: context.conversation_id,
})
console.log(meta.messageCount, meta.metadata?.title)
```

Python 示例:

```
meta = await context.store.get_conversation(
    conversation_id=context.conversation_id,
)
print(meta.message_count, (meta.metadata or {}).get("title"))
```

`listConversations` / `list_conversations`

列对话，按 `lastMessageAt` 排序，支持游标分页与按用户过滤。

参数

| Parameter                                  | Type                          | Required | Description                                       |
|--------------------------------------------|-------------------------------|----------|---------------------------------------------------|
| <code>limit</code>                         | <code>number</code>           | No       | 单页条数，默认 <code>20</code> ，范围 <code>[1, 100]</code> |
| <code>order</code>                         | <code>'asc' \   'desc'</code> | No       | 排序方向，默认 <code>'desc'</code> （最新优先）                |
| <code>after</code>                         | <code>string</code>           | No       | 游标，传上一页返回的 <code>nextCursor</code>                |
| <code>before</code>                        | <code>string</code>           | No       | 游标，传上一页返回的 <code>previousCursor</code>            |
| <code>userId</code> / <code>user_id</code> | <code>string</code>           | No       | 仅列该用户名下的对话（命中用户索引）                                |

返回值

`ListConversationsResult` —— `{ items, nextCursor, previousCursor }`。

TS 示例:

```
const { items, nextCursor } = await context.store.listConversations({
  userId: 'u_123',
  limit: 20,
})
```



Python 示例：

```
result = await context.store.list_conversations(user_id="u_123", limit=20)
items, next_cursor = result.items, result.next_cursor
```

updateConversation / update\_conversation

浅合并 metadata：同 key 覆盖、不同 key 保留。

参数

| Parameter                        | Type                | Required | Description                         |
|----------------------------------|---------------------|----------|-------------------------------------|
| conversationId / conversation_id | string              | Yes      | 对话 ID                               |
| metadata                         | Record<string, any> | Yes      | 待合并字段；value 为 null / None 表示删除该 key |

返回值

ConversationMeta —— 合并后的对话元信息。

TS 示例：

```
await context.store.updateConversation({
  conversationId: context.conversation_id,
  metadata: { title: '产品咨询', tag: null }, // 设标题，删 tag
})
```

Python 示例：

```
await context.store.update_conversation(
  conversation_id=context.conversation_id,
  metadata={"title": "产品咨询", "tag": None},
)
```

deleteConversation / delete\_conversation

删除整个对话，同步清理消息索引、会话元信息和全局会话索引，不可恢复。

参数：

| Parameter                        | Type   | Required | Description |
|----------------------------------|--------|----------|-------------|
| conversationId / conversation_id | string | Yes      | 对话 ID       |

TS 示例：

```
await context.store.deleteConversation({
  conversationId: context.conversation_id,
})
```

Python 示例：

```
await context.store.delete_conversation(
    conversation_id=context.conversation_id,
)
```

**toAnthropicMessages / to\_anthropic\_messages**

把 `getMessages` 返回的消息列表转成 Anthropic Messages API 的 `messages` 字段格式。

参数

| Parameter             | Type                   | Required | Description                                                |
|-----------------------|------------------------|----------|------------------------------------------------------------|
| <code>messages</code> | <code>Message[]</code> | Yes      | <code>getMessages</code> / <code>get_messages</code> 的返回结果 |

返回值

`Array<{ role: string; content: unknown }>` —— 可直接作为 `anthropic.messages.create({ messages })` 的入参。

TS 示例：

```
const history = await context.store.getMessages({
  conversationId: context.conversation_id,
})
const resp = await anthropic.messages.create({
  model: 'claude-sonnet-4',
  max_tokens: 1024,
  messages: context.store.toAnthropicMessages(history),
})
```

Python 示例：

```
history = await context.store.get_messages(
    conversation_id=context.conversation_id,
)
resp = await anthropic_client.messages.create(
    model="claude-sonnet-4",
    max_tokens=1024,
```

```
messages=context.store.to_anthropic_messages(history),
)
```

## toOpenAIInput / to\_openai\_input

把 `getMessages` 返回的消息列表转成 OpenAI Chat Completions API 的 `messages` 字段格式，保留 `role ∈ user / assistant / system / tool`，`content` 透传。

### 参数

| Parameter             | Type                   | Required | Description                                                |
|-----------------------|------------------------|----------|------------------------------------------------------------|
| <code>messages</code> | <code>Message[]</code> | Yes      | <code>getMessages</code> / <code>get_messages</code> 的返回结果 |

### 返回值

`Array<{ role: string, content: any }>` —— 可直接作为 `openai.chat.completions.create({ messages })` 的入参。

### TS 示例：

```
const history = await context.store.getMessages({
  conversationId: context.conversation_id,
})
const resp = await openai.chat.completions.create({
  model: 'gpt-4o',
  messages: context.store.toOpenAIInput(history),
})
```

### Python 示例：

```
history = await context.store.get_messages(
    conversation_id=context.conversation_id,
)
resp = await openai_client.chat.completions.create(
    model="gpt-4o",
    messages=context.store.to_openai_input(history),
)
```

## 数据结构

```
interface Message {
  messageId: string // msg_xxx (appendMessage 自动生成)
  role: 'user' | 'assistant' | 'system' | 'tool'
  content: any // string / array / object (多模态)
  createdAt: number // ms 时间戳
```

```
metadata?: Record<string, any>          // 自定义 (token 数、tool_call 等)
updatedAt?: number                      // 仅 updateMessage 后存在
}

interface ConversationMeta {
  conversationId: string
  createdAt: number                    // ms 时间戳
  lastMessageAt: number               // ms 时间戳
  messageCount: number
  metadata?: Record<string, any>      // 业务自定义 (标题、用户、标签等)
}

interface ListConversationsResult {
  items: ConversationMeta[]
  nextCursor?: string                 // 下一页游标, 传给 after 参数
  previousCursor?: string              // 上一页游标, 传给 before 参数
}
```

Python 端字段为 `message_id / conversation_id / created_at / last_message_at / message_count / next_cursor / previous_cursor`，整体形态一致。

## 限制与配额

| 项                               | 默认            | 超限行为                                    |
|---------------------------------|---------------|-----------------------------------------|
| <code>conversation_id</code> 长度 | ≤ 256 字符      | 抛 <code>MemoryValidationError</code>    |
| 单条 <code>content</code> 大小      | ≤ 50MB (序列化后) | 抛 <code>MemoryValidationError</code>    |
| 单对话最大消息数                        | 10000         | 抛 <code>MemoryQuotaExceededError</code> |
| <code>limit</code> 上限           | 100           | 抛 <code>MemoryValidationError</code>    |
| <code>limit</code> 下限           | 1             | 抛 <code>MemoryValidationError</code>    |

单条 `content` 超大（长文档、图片原始数据）建议放对象存储，传 URL 进消息。

## 对话摘要

`getMessages` 单次最多取 100 条，长对话下推荐做法是保留最近 N 条原文 + 早期内容压成一段摘要，每次请求时从 Store 拼装回 prompt。Store 本身不调 LLM、不生成摘要，只提供 `ConversationMeta.metadata` 这个 JSON 容器。

## 职责切分

| 谁负责       | 做什么                                                                                                                                           |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Store（已有） | 提供 <code>metadata</code> JSON 字段 + 浅合并写入。约定两个 key： <code>summary</code> （摘要文本）、 <code>summarizedUntil</code> （已摘要到哪条 <code>messageId</code> ） |
| 业务侧（自己写）  | 决定何时摘要、调便宜模型生成摘要、调 <code>updateConversation</code> 写回                                                                                         |

`summary` / `summarizedUntil` 是约定 key 而非 schema，Store 只把它当任意 JSON 存。你可以换名字，只要业务侧前后保持一致。

### 实现示例

可以使用便宜模型做摘要——压缩 + 保关键事实不需要主模型那种推理能力。prompt 中四条要求都不能省，否则会出现摘要发散、寒暄混入、语言乱跳、越摘越长等问题。

```
async function summarizeWithLLM(previousSummary, newMessages) {
  const transcript = newMessages
    .filter(m => m.role === 'user' || m.role === 'assistant')
    .map(m => `${m.role}: ${
      typeof m.content === 'string' ? m.content : JSON.stringify(m.content)
    }`)
    .join('\n')
  const prompt = previousSummary
    ? `下面是一段对话的【已有摘要】和【新增对话】，请合并成一段新摘要。`
    : `
要求：
- 保留：关键事实、用户偏好、未完成任务、重要的上下文设定
- 删除：寒暄、确认收到、重复内容、工具调用细节
- 用与最近对话相同的语言书写
- 控制在 500 字内

【已有摘要】
${previousSummary}

【新增对话】
${transcript}

【新摘要】`
    : `
要求：
- 保留：关键事实、用户偏好、未完成任务、重要的上下文设定
- 删除：寒暄、确认收到、重复内容、工具调用细节
- 用与对话相同的语言书写
- 控制在 500 字内

【对话】`
}
```

`${transcript}`

`【摘要】``

```
const { text } = await generateText({
```

```
  model: openai('gpt-4o-mini'),
```

```
  prompt,
```

```
  maxTokens: 800,
```

```
})
```

```
return text.trim()
```

```
}
```